

CSS COMPLETE NOTES

BASIC → INTERMEDIATE → ADVANCED

Proper handwritten-notes style study material

□ Har concept ko is order mein padha gaya hai: Definition → Why → Types → Syntax → Implementation → Example → Diagram/working → Important points → Common mistakes → Practice.

Target: CSS ko sirf properties ki list ke roop mein nahi, balki page layout ko logically build karna seekhna.

Chapter 1 — CSS Introduction

Proper Definition: CSS stands for Cascading Style Sheets. It is a stylesheet language used to describe how HTML elements should look and how they should be arranged on a webpage. CSS controls colors, fonts, spacing, borders, dimensions, layouts, animations and responsive behavior. CSS does not replace HTML; HTML provides the structure and CSS provides the presentation of that structure.

Why CSS is needed

- HTML alone can create structure, but CSS makes the structure visually organized and usable.
- One CSS rule can style many elements, reducing repeated formatting.
- CSS separates content from presentation, so changing a design does not require rewriting HTML.
- CSS supports responsive layouts for phones, tablets and desktops.

CSS working — simple flow

```
HTML file → Browser parses HTML → DOM
CSS file → Browser parses CSS → CSS rules match DOM
↓
Style + Layout + Paint
↓
Web page
```

Advantages of CSS

- Consistency: same design can be reused across pages.
- Maintainability: styles can live in separate files.
- Performance: browser can cache external CSS.
- Responsive design: layout can adapt to screen conditions.

Basic vocabulary

Term	Meaning
Selector	Chooses the HTML element(s) to style.
Property	The feature being changed, e.g. color.
Value	The setting given to a property, e.g. red.
Declaration	A property-value pair such as color:red;
Rule set	Selector plus one or more declarations.

Chapter 2 — CSS Syntax and First Implementation

Proper Definition: CSS syntax is the standard structure used to write a CSS rule. A selector is followed by curly braces. Inside the braces, declarations are written as property-value pairs separated by a colon and normally terminated with a semicolon.

Syntax

```
selector {  
  property: value;  
  property: value;  
}
```

Complete example

```
h1 {  
  color: blue;  
  font-size: 32px;  
  text-align: center;  
}
```

How browser reads it

- h1 = select all h1 elements.
- color = choose the text-color property.
- blue = value assigned to color.
- font-size:32px = another declaration.
- Each declaration changes a specific aspect of the selected element.

CSS comments

```
/* This comment is ignored by the browser */  
.card { padding: 20px; }
```

- Comments are useful for explaining sections of a stylesheet and do not appear on the webpage.

Common syntax mistakes

- Missing semicolon can make later declarations fail or behave unexpectedly.
- Writing box { } when HTML uses class="box" — class selector must be .box.
- Writing #box for a class or .box for an ID.

Chapter 3 — Ways to Apply CSS

Proper Definition: There are three traditional ways to apply CSS to HTML: inline CSS, internal CSS and external CSS. They differ mainly in where the CSS is written and how reusable it is.

Type 1 — Inline CSS

```
<button style="background:black; color:white;">Save</button>
```

Definition: Inline CSS is written directly inside an HTML element's style attribute. It affects that particular element. It is useful for a very small one-off style, but becomes difficult to maintain when used everywhere.

Type 2 — Internal CSS

```
<head>
<style>
p { color: green; }
</style>
</head>
```

Definition: Internal CSS is written inside a style element, usually in the HTML head. It is convenient when styling one page.

Type 3 — External CSS

```
<!-- index.html -->
<link rel="stylesheet" href="style.css">

/* style.css */
p { color: green; }
```

Definition: External CSS is stored in a separate .css file and connected with link. It is generally the cleanest approach for multi-page projects.

Comparison

Feature	Inline	Internal	External
Reusable	Low	Within page	High
Maintenance	Difficult	Medium	Easy
Best for	One-off style	Single page	Real projects

Recommended folder structure

```
my-site/
index.html
css/
style.css
images/
js/
```

Chapter 4 — CSS Selectors

Proper Definition: A CSS selector is a pattern used to identify which HTML element or elements should receive a style. Selectors are the foundation of CSS because every declaration must be applied to some target.

1. Universal selector

```
* {  
  box-sizing: border-box;  
}
```

Definition: Selects every element on the page.

2. Element selector

```
p { color: #333; }  
h1 { font-size: 2rem; }
```

Definition: Selects all elements of a given HTML tag.

3. Class selector

```
.card { padding: 20px; }  
  
<div class="card">Content</div>
```

Definition: Selects elements having the specified class. A class can be reused on many elements.

4. ID selector

```
#header { height: 70px; }  
  
<header id="header">...</header>
```

Definition: Selects the element with the specified ID. IDs are normally unique within a document.

5. Grouping selector

```
h1, h2, p {  
  font-family: Arial, sans-serif;  
}
```

6. Attribute selector

```
input[type="email"] {  
  border: 1px solid gray;  
}
```

Definition: Selects elements according to an attribute or attribute value.

7. Relationship selectors

```
.card p { ... } /* descendant */  
.card > p { ... } /* direct child */  
h2 + p { ... } /* adjacent sibling */  
h2 ~ p { ... } /* general sibling */
```

Selector diagram

```
HTML  
■■■ .card  
■■■ h2  
■■■ p  
  
.card p → targets p anywhere inside .card  
.card > p → targets only direct p child
```

Important

- Use classes for reusable component styling.
- Avoid very long selector chains because they become hard to override.

- Selector correctness matters more than adding more CSS.

Chapter 5 — Colors

Proper Definition: CSS color properties specify the color of text, borders, backgrounds and other visual effects. CSS supports named colors and several numeric color models.

Color types / formats

Format	Example	Meaning
Name	red	Predefined color keyword
HEX	#ff0000	Hexadecimal RGB value
RGB	rgb(255,0,0)	Red, green, blue channels
RGBA	rgba(255,0,0,.5)	RGB plus alpha transparency
HSL	hsl(0,100%,50%)	Hue, saturation, lightness
HSLA	hsla(0,100%,50%,.5)	HSL plus alpha

Implementation

```
h1 { color: #222; }  
.box { background-color: rgb(245,245,245); }  
.overlay { background-color: rgba(0,0,0,.5); }
```

Foreground vs background

- color normally changes foreground/text color.
- background-color changes the background of the element.
- border-color changes border color.

Chapter 6 — Backgrounds

Proper Definition: Background properties control the visual layer behind an element. A background can be a solid color, image, gradient or a combination of layers.

Background color

```
body { background-color: #f5f5f5; }
```

Background image

```
.hero {  
  background-image: url("hero.jpg");  
  background-repeat: no-repeat;  
  background-position: center;  
  background-size: cover;  
}
```

Important background-size values

- cover: scales the image to cover the whole box; some cropping may occur.
- contain: scales the image so the whole image fits; empty space may remain.
- auto: keeps the image's natural sizing behavior.

Gradients

```
background: linear-gradient(90deg, black, gray);  
background: radial-gradient(circle, white, gray);
```

Multiple backgrounds

```
.box {  
  background-image: url("pattern.png"), linear-gradient(fff,ddd);  
  background-position: center, center;  
  background-size: cover, cover;  
}
```

Chapter 7 — Text and Fonts

Proper Definition: Typography in CSS controls the visual appearance, spacing and readability of text. Text properties affect the content itself, while font properties control the typeface and weight.

Text properties

```
p {  
  color:#222;  
  text-align:center;  
  text-decoration:underline;  
  text-transform:uppercase;  
  line-height:1.6;  
  letter-spacing:1px;  
  word-spacing:4px;  
}
```

Font properties

```
body {  
  font-family: Arial, sans-serif;  
  font-size: 16px;  
  font-weight: 400;  
  font-style: normal;  
}
```

Font family and fallback

A font stack lists alternatives. If the first font is unavailable, the browser tries the next one. The final generic family is a safety fallback.

Useful text control

```
white-space: nowrap;  
overflow: hidden;  
text-overflow: ellipsis;
```

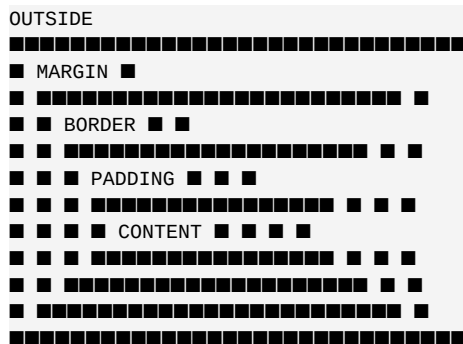
Text shadow

```
h1 { text-shadow: 2px 2px 5px #999; }
```

Chapter 8 — Box Model

Proper Definition: The CSS box model describes how an element occupies space. Every element can be understood as four layers: content, padding, border and margin.

Full diagram



Content

Definition: The content area contains the actual text, image, child elements or other content of the box.

Padding

Definition: Padding is internal space between content and border.

```
.box { padding: 20px; }  
.box { padding: 10px 20px; }  
/* top-bottom 10px, left-right 20px */
```

Border

Definition: Border is the visible boundary around padding/content.

```
.box { border: 2px solid black; }  
.box { border-radius: 12px; }
```

Margin

Definition: Margin is external space outside the border and separates the element from neighboring elements.

```
.box { margin: 20px; }  
.box { margin: 0 auto; }
```

box-sizing

Definition: box-sizing decides how declared width and height are calculated.

```
* { box-sizing: border-box; }
```

- content-box: declared width normally applies to content; padding and border are added outside it.
- border-box: declared width includes content + padding + border.

Calculation example

```
width:300px;  
padding:20px;  
border:5px solid black;  
box-sizing:border-box;
```

Total outer width = 300px

Chapter 9 — Width, Height and Units

Proper Definition: CSS units define measurements used for dimensions, spacing, typography and layout. Some units are fixed-like while others are relative to a parent, root font size or viewport.

Major units

Unit	Definition	Example
px	CSS pixel unit	20px
%	Relative to a containing/reference size	50%
em	Relative to the relevant font-size context	1.5em
rem	Relative to root element's font size	1rem
vw	1% of viewport width	50vw
vh	1% of viewport height	50vh
vmin	1% of smaller viewport dimension	10vmin
vmax	1% of larger viewport dimension	10vmax
fr	Fraction of available Grid space	1fr

Width controls

```
.container {  
width:90%;  
max-width:1100px;  
min-width:280px;  
}
```

Height controls

```
.hero {  
min-height:70vh;  
height:auto;  
max-height:900px;  
}
```

Modern functions

```
width: calc(100% - 40px);  
width: min(90%, 1000px);  
width: max(300px, 50%);  
font-size: clamp(1rem, 2vw, 2rem);
```

Practical choice

- rem: scalable text and spacing.
- %: fluid sizing relative to a containing context.
- vw/vh: viewport-based sizing.
- fr: Grid columns/rows.
- px: borders and small fixed details are common uses.

Chapter 10 — Display

Proper Definition: The display property controls an element's outer and inner layout behavior. It determines whether the element participates as a block, inline item, flex container, grid container, or is removed from layout.

block

Definition: A block-level box normally starts on a new line and can generally accept width and height.

```
div { display:block; }
```

inline

Definition: An inline box participates within the text flow. Width and height do not behave like a normal block box.

```
span { display:inline; }
```

inline-block

Definition: It flows inline but behaves like a block container for sizing.

```
.badge { display:inline-block; width:100px; }
```

flex and grid

```
.nav { display:flex; }  
.cards { display:grid; }
```

none vs visibility

```
.a { display:none; }  
.b { visibility:hidden; }
```

- display:none removes the element from layout.
- visibility:hidden hides it but normally preserves its layout space.

Overflow

```
overflow:hidden;  
overflow:auto;  
overflow-x:auto;  
overflow-y:auto;
```


Chapter 12 — Flexbox

Proper Definition: Flexbox is a one-dimensional CSS layout system. It arranges direct children of a flex container along a main axis and a cross axis, and provides powerful alignment and sizing controls.

Start flex

```
.container { display:flex; }
```

Axes diagram

```
flex-direction: row
MAIN AXIS ████████████████████████████████████████→
██████████████████████████████████████████████████████
█ [1] [2] [3] █
██████████████████████████████████████████████████████
↓ CROSS AXIS
```

flex-direction

```
row;
row-reverse;
column;
column-reverse;
```

Definition: Sets the direction of the main axis.

justify-content

Definition: Aligns/distributes flex items along the main axis.

```
justify-content:flex-start;
justify-content:center;
justify-content:space-between;
justify-content:space-around;
justify-content:space-evenly;
```

align-items

Definition: Aligns flex items on the cross axis.

```
align-items:flex-start;
align-items:center;
align-items:flex-end;
align-items:stretch;
```

flex-wrap and gap

```
flex-wrap:wrap;
gap:20px;
```

Child sizing

```
.item {
flex-grow:1;
flex-shrink:1;
flex-basis:0;
}
/* shorthand */
.item { flex:1; }
```

align-self

```
.special { align-self:center; }
```

Complete navbar example

```
.navbar {
display:flex;
justify-content:space-between;
align-items:center;
gap:20px;
}
```

Golden rule

□ justify-content = main axis. align-items = cross axis. Main axis changes when flex-direction changes.

Chapter 14 — Borders, Radius, Shadows and Overflow

Proper Definition: These properties define boundaries, corner shape, visual depth and how content that exceeds a box is handled.

Border

```
border:1px solid #ccc;  
border-width:2px;  
border-style:dashed;  
border-color:#333;
```

Border radius

```
border-radius:12px;  
border-radius:50%;
```

Box shadow

```
box-shadow:0 8px 24px rgba(0,0,0,.12);
```

Shadow anatomy

Common order: horizontal offset → vertical offset → blur radius → optional spread → color.

```
box-shadow: 0 4px 12px 0 rgba(0,0,0,.15);
```

Overflow

```
overflow:hidden;  
overflow:auto;  
overflow-x:auto;  
overflow-y:auto;
```

Card implementation

```
.card {  
border:1px solid #ddd;  
border-radius:16px;  
padding:20px;  
box-shadow:0 8px 24px rgba(0,0,0,.08);  
overflow:hidden;  
}
```

Chapter 15 — Lists and Tables

Proper Definition: CSS can control list markers and table borders, spacing, alignment and visual hierarchy.

Lists

```
ul { list-style-type:square; }
ol { list-style-type:decimal; }
ul { list-style-position:inside; }
```

Remove default bullets

```
nav ul {
  list-style:none;
  padding:0;
  margin:0;
}
```

Tables

```
table {
  width:100%;
  border-collapse:collapse;
}
th,td {
  border:1px solid #ccc;
  padding:10px;
  text-align:left;
}
```

Row styling

```
tbody tr:nth-child(even) {
  background:#f7f7f7;
}
```

Rule

□ Table ko tabular data ke liye use karo. Page layout banane ke liye CSS Grid/Flexbox use karo.

Chapter 16 — Forms

Proper Definition: Form styling improves readability, interaction and consistency of labels, inputs, buttons, selects and textareas.

Basic form CSS

```
label {
display:block;
margin-bottom:6px;
}
input, select, textarea {
width:100%;
padding:10px;
border:1px solid #ccc;
border-radius:8px;
}
```

States

```
input:focus { outline:2px solid #555; }
button:hover { transform:translateY(-1px); }
button:active { transform:translateY(0); }
input:disabled { opacity:.6; }
input:checked { accent-color:#333; }
input:required { ... }
```

Why :focus matters

Keyboard users need a visible indication of which control is currently focused. Removing outlines without providing an equivalent visible focus style harms usability.

Button example

```
.btn {
display:inline-flex;
align-items:center;
justify-content:center;
padding:10px 16px;
border:0;
border-radius:8px;
cursor:pointer;
}
```

Chapter 17 — Pseudo-classes

Proper Definition: A pseudo-class is a keyword added to a selector to target an element based on a state, condition or structural position rather than adding another class in HTML.

Common state pseudo-classes

```
:hover
:focus
:active
:visited
:disabled
:checked
:required
:valid
:invalid
```

Structural pseudo-classes

```
:first-child
:last-child
:nth-child(2n)
:nth-child(odd)
:nth-of-type(2)
:not(.active)
```

Example

```
li:first-child { font-weight:bold; }
li:nth-child(even) { background:#f5f5f5; }
button:hover { opacity:.85; }
```

Pseudo-class vs class

- Class: you explicitly add a class in HTML, e.g. `.active`.
- Pseudo-class: browser determines the state/relationship, e.g. `:hover`.

Chapter 18 — Pseudo-elements

Proper Definition: A pseudo-element represents a conceptual part of an element, allowing CSS to style content such as the first letter, first line, or generated decorative content.

Syntax

```
selector::pseudo-element { property:value; }
```

Common pseudo-elements

```
::before  
::after  
::first-letter  
::first-line  
::selection  
::placeholder
```

before / after

```
.card::before {  
  content:"";  
  position:absolute;  
  inset:0;  
}  
.card::after {  
  content:"New";  
}
```

Important rule

- `::before` and `::after` generally need content to generate their box.
- Use generated content mainly for decoration; important semantic information should exist in HTML.

Chapter 19 — Transforms

Proper Definition: The transform property changes an element's visual geometry without changing the normal document flow in the same way as layout properties do.

Translate

```
transform:translateX(20px);
transform:translateY(10px);
transform:translate(20px,10px);
```

Scale

```
transform:scale(1.1);
transform:scaleX(1.2);
```

Rotate

```
transform:rotate(10deg);
```

Skew

```
transform:skewX(10deg);
transform:skewY(5deg);
```

Transform origin

```
transform-origin:center;
transform-origin:top left;
```

Combined transform

```
transform:translateY(-4px) scale(1.02) rotate(.5deg);
```

Chapter 20 — Transitions

Proper Definition: A transition creates a smooth interpolation between two computed states when a property changes.

Basic

```
.btn {  
  transition:transform .2s ease, background-color .2s ease;  
}  
.btn:hover {  
  transform:translateY(-2px);  
}
```

Four important parts

Part	Meaning
property	Which property should animate
duration	How long the change takes
timing-function	How speed changes over time
delay	How long before transition begins

Timing functions

```
ease  
ease-in  
ease-out  
linear  
cubic-bezier(...)
```

Mistake

- transition ko :hover ke andar rakhna usually wrong behavior deta hai; transition ko base state par rakhna generally smoother hota hai.

Chapter 21 — Animations

Proper Definition: CSS animation uses @keyframes to define a sequence of styles and animation properties to control how that sequence runs.

@keyframes

```
@keyframes slide {  
  from { transform:translateX(0); }  
  to { transform:translateX(100px); }  
}
```

Apply animation

```
.box {  
  animation:slide 1s ease infinite alternate;  
}
```

Important properties

- animation-name — keyframes name.
- animation-duration — duration.
- animation-delay — start delay.
- animation-iteration-count — repetitions.
- animation-direction — normal/reverse/alternate etc.
- animation-fill-mode — styles before/after run.
- animation-play-state — running/paused.

Multi-step

```
@keyframes pulse {  
  0% { transform:scale(1); }  
  50% { transform:scale(1.05); }  
  100% { transform:scale(1); }  
}
```

Reduced motion

```
@media (prefers-reduced-motion: reduce) {  
  *,*::before,*::after {  
    animation:none !important;  
    transition:none !important;  
  }  
}
```

Chapter 22 — Responsive Web Design

Proper Definition: Responsive web design means designing a webpage so its layout and content adapt to different viewport sizes and device conditions instead of assuming one fixed screen size.

Viewport meta tag

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Media query

```
@media (max-width:768px) {  
  .navbar { flex-direction:column; }  
}
```

Mobile-first approach

Definition: Start with the small-screen base layout and add enhancements for larger screens using min-width media queries.

Example

```
.container {  
  width:92%;  
  margin-inline:auto;  
}  
@media (min-width:768px) {  
  .container { width:85%; }  
}  
@media (min-width:1200px) {  
  .container { max-width:1100px; }  
}
```

Responsive checklist

- Avoid unnecessary fixed widths.
- Use flexible Grid/Flexbox layouts.
- Images should normally use max-width:100%; height:auto.
- Test at multiple widths, not only one phone and one desktop size.

Chapter 23 — CSS Variables

Proper Definition: CSS custom properties, commonly called CSS variables, store reusable values that can be referenced with `var()`. They make themes, spacing systems and repeated design values easier to manage.

Define variables

```
:root {  
  --primary:#222;  
  --space:16px;  
  --radius:12px;  
}
```

Use variables

```
.card {  
  color:var(--primary);  
  padding:var(--space);  
  border-radius:var(--radius);  
}
```

Fallback value

```
color:var(--primary, black);
```

Local variables

```
.dark {  
  --text:white;  
  --bg:#111;  
}  
.dark .card {  
  color:var(--text);  
  background:var(--bg);  
}
```

Benefits

- Centralized design tokens.
- Easy theme changes.
- Less repeated values.
- Useful for component systems.

Chapter 24 — CSS Functions

Proper Definition: CSS functions are built-in expressions that calculate, select or generate values. They make layouts more fluid and reduce the need for many breakpoint-specific rules.

calc()

```
width:calc(100% - 40px);  
height:calc(100vh - 80px);
```

min()

```
width:min(90%,1000px);
```

max()

```
width:max(300px,50%);
```

clamp()

```
font-size:clamp(1rem,2vw,2rem);
```

repeat()

```
grid-template-columns:repeat(3,1fr);
```

minmax()

```
grid-template-columns:repeat(auto-fit,minmax(220px,1fr));
```

var()

```
color:var(--primary);
```

Chapter 25 — Cascade, Specificity and Inheritance

Proper Definition: The cascade is the process CSS uses to determine which declarations apply when several rules match an element. Specificity is one factor in that process, while inheritance allows some properties to come from an ancestor.

Specificity example

```
p { color:blue; }
.card p { color:green; }
#main p { color:red; }
```

Specificity levels — practical model

- Inline styles generally have very high specificity in normal author CSS.
- ID selectors are stronger than class/attribute/pseudo-class selectors.
- Class, attribute and pseudo-class selectors are stronger than element selectors.
- If specificity is otherwise equal, later source order commonly wins.

Inheritance

Some properties such as color and font-family commonly inherit. Layout properties such as margin and padding normally do not inherit.

```
body { color:#222; font-family:Arial; }
p { /* can inherit both */ }
```

!important

```
color:red !important;
```

Definition: !important raises the priority of a declaration against normal declarations. It should be used sparingly because it makes the cascade harder to reason about.

Debugging specificity

- Inspect element → Styles → find crossed-out declaration → identify which selector won → simplify specificity if possible.

Chapter 26 — Advanced Selectors

Proper Definition: Advanced selectors allow precise targeting based on attributes, relationships and structural conditions. They can reduce unnecessary classes when used carefully.

Attribute operators

```
a[href^="https"] { ... } /* starts with */  
a[href$=".pdf"] { ... } /* ends with */  
a[href*="example"] { ... } /* contains */
```

:not()

```
.menu li:not(.active) { opacity:.7; }
```

:is()

```
:is(h1,h2,h3) { line-height:1.2; }
```

:where()

```
:where(.card,.panel) { border-radius:12px; }
```

:has()

```
.card:has(img) { padding-top:0; }
```

:nth-child()

```
li:nth-child(2n) { background:#f5f5f5; }  
li:nth-child(3n+1) { ... }
```

Caution

- Advanced selectors are powerful, but readability matters. A complicated selector is not automatically better CSS.

Chapter 27 — Modern CSS: Nesting, Container Queries and Logical Properties

Proper Definition: Modern CSS provides features for component-oriented styling and responsive behavior that depends on the component's own container rather than only the viewport.

CSS nesting

```
.card {  
  padding:20px;  
  & h2 { margin:0; }  
  &:hover { transform:translateY(-2px); }  
}
```

Container queries

```
.card-wrapper { container-type:inline-size; }  
@container (min-width:500px) {  
  .card { display:flex; }  
}
```

Container vs media query

- @media responds to conditions such as viewport size.
- @container responds to the size/condition of a query container, which is useful for reusable components.

Logical properties

```
margin-inline:auto;  
padding-inline:20px;  
margin-block:16px;  
inset-inline-start:0;
```

Why logical properties?

They describe directions in a writing-mode-aware way and are useful when interfaces may support different writing directions.

Chapter 28 — Reusable Layout Patterns

Proper Definition: Professional CSS is largely about solving recurring layout problems with simple, reusable patterns rather than writing one-off styles for every element.

Centered container

```
.container {  
  width:min(92%,1100px);  
  margin-inline:auto;  
}
```

Navbar

```
.navbar {  
  display:flex;  
  align-items:center;  
  justify-content:space-between;  
  gap:1rem;  
}
```

Card grid

```
.cards {  
  display:grid;  
  grid-template-columns:repeat(auto-fit,minmax(220px,1fr));  
  gap:1.25rem;  
}
```

Hero

```
.hero {  
  min-height:70vh;  
  display:grid;  
  place-items:center;  
  text-align:center;  
}
```

Reusable button

```
.btn {  
  display:inline-flex;  
  align-items:center;  
  justify-content:center;  
  gap:.5rem;  
  padding:.7rem 1rem;  
  border-radius:.6rem;  
}
```

Chapter 29 — DevTools and Debugging

Proper Definition: CSS debugging is the process of finding why the browser's rendered result differs from the intended design. Browser DevTools is the main practical tool for this job.

Debugging workflow

- Right-click → Inspect.
- Select the element.
- Read the Styles panel.
- Crossed-out rule means another rule is winning or the declaration is not applicable.
- Check Computed styles.
- Open the Box Model view to inspect margin, border, padding and dimensions.

Common bugs

- Wrong selector: box vs .box.
- Class/ID spelling mismatch.
- External CSS path is wrong.
- A more specific selector overrides the rule.
- Parent's display/position affects child layout.
- Unexpected overflow caused by sizing or long content.

Debug outline

```
* { outline:1px solid red; }
```

Temporary trick only. Remove it after debugging.

Best question to ask

□ Instead of randomly changing values, ask: Which element is wrong? Which parent controls its layout? Which CSS rule is currently winning? What does the Box Model say?

Chapter 30 — CSS Architecture and Best Practices

Proper Definition: CSS architecture means organizing styles so a project stays understandable, reusable and predictable as it grows.

Good naming

- Use names based on role/component: `.card`, `.navbar`, `.btn-primary`.
- Avoid meaningless names such as `.red-box` unless color itself is the purpose.

Avoid over-specific selectors

```
/* Hard to maintain */
body main section .container div ul li a { ... }

/* Better */
.nav-link { ... }
```

Use variables for repeated design tokens

```
:root {
  --space-sm: 8px;
  --space-md: 16px;
  --radius-md: 12px;
}
```

Possible file structure

```
css/
base.css
layout.css
components.css
utilities.css
responsive.css
```

Avoid

- Hundreds of `!important` declarations.
- Styling everything through IDs.
- Huge duplicated blocks.
- Randomly mixing component and global styles.

Chapter 31 — Accessibility and Performance

Proper Definition: Accessible CSS helps people with different abilities use the interface. Performance-conscious CSS keeps styles manageable and avoids unnecessary work.

Accessibility rules

- Keep visible keyboard focus styles.
- Do not communicate important information through color alone.
- Use readable text and adequate contrast.
- Respect reduced-motion preferences.
- Keep interactive controls usable on touch devices.

Reduced motion

```
@media (prefers-reduced-motion:reduce) {  
  *,*::before,*::after {  
    animation:none !important;  
    transition:none !important;  
    scroll-behavior:auto !important;  
  }  
}
```

Performance basics

- Remove unused CSS in production.
- Avoid unnecessary duplication.
- Keep selectors understandable.
- Large images are primarily an asset optimization issue; CSS alone does not make oversized images small.

Chapter 32 — Complete Mini Projects

Proper Definition: The fastest way to turn CSS knowledge into skill is to implement complete interfaces where several concepts work together.

Project 1 — Profile Card

- Use: box model + typography + border-radius + shadow + hover transition.

```
.card { max-width:320px; padding:20px; border-radius:16px; }  
.card img { width:100%; border-radius:12px; }  
.card:hover { transform:translateY(-4px); }
```

Project 2 — Login Form

- Use: labels + inputs + :focus + button + responsive width.

Project 3 — Navbar

- Use: Flexbox + gap + hover + mobile media query.

Project 4 — Portfolio

- Header → Hero → About → Skills → Projects → Contact → Footer.
- Use Flexbox for navigation and Grid for project cards.

Project 5 — Learning Notes Website

- Use reusable note cards, responsive Grid, variables, pseudo-classes and transitions. Later add JavaScript for search/filter.

Professional build order

1. HTML structure
2. Base/reset styles
3. Layout
4. Spacing
5. Typography
6. Colors/borders/shadows
7. Responsive design
8. Interaction states
9. Accessibility
10. Cleanup/refactor

Chapter 33 — Final Revision Sheet

Proper Definition: This chapter condenses the most important CSS concepts for quick revision and practical recall.

Must remember

- Selector → Property → Value.
- Box model → Content + Padding + Border + Margin.
- Flexbox → one-dimensional.
- Grid → two-dimensional.
- justify-content → main axis.
- align-items → cross axis.
- position:absolute → removed from normal flow and positioned against its containing block.
- display:none → removed from layout.
- rem → root font-size based.
- @media → responsive conditions; @container → container conditions.
- Specificity + cascade + source order resolve competing declarations.

Property cheat sheet

Category	Important properties
Text	color, text-align, text-decoration, text-transform, line-height, letter-spacing
Font	font-family, font-size, font-weight, font-style
Box	width, height, padding, border, margin, box-sizing
Background	background-color, background-image, background-size, background-position
Flex	display, flex-direction, justify-content, align-items, flex-wrap, gap, flex
Grid	grid-template-columns, grid-template-rows, gap, grid-area, place-items
Position	position, inset, top/right/bottom/left, z-index
Motion	transform, transition, animation, @keyframes
Responsive	@media, @container
Advanced	var(), calc(), min(), max(), clamp(), :is(), :where(), :has()

Final practice challenge

Build a responsive portfolio website using:

- ✓ External CSS
- ✓ CSS variables
- ✓ Box model
- ✓ Flexbox navbar
- ✓ Grid project cards
- ✓ Pseudo-classes
- ✓ Transition
- ✓ Animation
- ✓ Media query
- ✓ Accessible focus state
- ✓ Mobile-first layout

Final rule

□ Agar tum kisi CSS property ka meaning, use-case aur effect browser DevTools mein explain kar sakte ho, tab tumne us concept ko actually seekha hai. Sirf syntax yaad hona mastery nahi hai.